CS5730 Operating Systems: Past Exam Solutions

All past papers in the repo (2011, 2013, 2019, 2020, 2021, 2022), in order. Each entry gives the year, the full question as printed, then a model answer sized to fit the space on the exam paper (prose kept to about 250 words; code, tables and diagrams added where the question needs them). Repeated questions use the same answer every time.

Year	Paper	Questions
2011	CS5730 Operating Systems Midterm (2011/11/14)	20 answers
2013	CS5730 Operating Systems Midterm (2013/11/11)	16 answers
2019	CS5730 Operating Systems Midterm (2019/12/12)	11 answers
2020	CS5730 Operating Systems (2020/12/1; the paper is titled "Final")	16 answers
2021	CS5730 Operating Systems Midterm (2021/11/30)	16 answers
2022	CS5730 Operating Systems Midterm (2022/11/22)	11 answers

2011 · CS5730 Operating Systems Midterm (2011/11/14)

1. Short Answer: Your answers should be as concise as possible; aim for three or four sentences.

Q1.1 (5 pt) Explain why implementing synchronization primitives by disabling interrupts is not appropriate in a single-processor system if the synchronization primitives are to be used in user-level programs. 2011

ANSWER

Disabling interrupts works on a uniprocessor only for **trusted kernel code**. It is the wrong tool for **user-level** synchronization because:

1. **It is a privileged instruction.** Letting user programs execute it breaks dual-mode protection; the hardware traps if user mode tries.
2. **The OS loses control of the CPU.** While interrupts are off, the **timer interrupt** cannot fire, so no preemption or context switch can happen. A buggy or malicious program could disable interrupts and never re-enable them (e.g. an infinite loop in its critical section). It would then **monopolize the CPU** and hang the whole system.
3. **Other events are delayed or lost.** I/O-completion interrupts and clock ticks are postponed, so devices sit idle, the system clock drifts and response time suffers. User critical sections can be long, which makes this worse.

(It also does not scale: on a multiprocessor it only stops the local CPU.) User programs should instead use OS-provided semaphores/mutexes or atomic hardware instructions such as test_and_set or swap.

Q1.2 (5 pt) Provide an example in which multithreading does not provide better performance than a single-threaded solution. 2011

ANSWER

Multithreading helps only if there is work that can overlap. Two examples where it does not help:

- A **strictly sequential computation**, where each step depends on the previous result. Examples are computing a tax return or a recurrence such as Fibonacci with an iterative loop. There is nothing to run in parallel, and threads only add creation, context-switch and synchronization overhead.
- A **CPU-bound program on a single processor** with no I/O (e.g. a matrix multiply). The threads just take turns on the same CPU, so the total time is the same or worse because of switching overhead.

Another standard example is a shell or interactive program that only waits for one user's input. It spends its time blocked anyway.

Q1.3 (5 pt) Show that, if the `wait()` and `signal()` semaphore operations are not executed atomically, then mutual exclusion may be violated. 2011

ANSWER

Busy-waiting definitions: `wait(S){ while (S <= 0); S--; }` and `signal(S){ S++; }`, with S = 1 (unlocked).

If wait() is not atomic, the test and the decrement can be interleaved:

Step	P0	P1	S
1	tests S<=0 → false (S=1)		1
2	*preempted*	tests S<=0 → false (S=1)	1
3		S-- ; enters critical section	0
4	S-- ; enters critical section		−1

Both P0 and P1 are in the critical section, so mutual exclusion is violated. Likewise, S-- and S++ are each load, modify, store at machine level. If two processes run them concurrently, an update can be lost, e.g. two signal() calls increment S only once. A lost signal can block a process forever, and an extra count lets two processes in. So wait() and signal() must be indivisible.

2. Consider a multiprocessor system and a multithreaded program written using the many-to-many threading model. Let the number of user-level threads in the program be more than the number of processors in the system. Discuss the performance implications of the following scenarios.

Q2.1 (5 pt) The number of kernel threads allocated to the program is less than the number of processors. 2011

ANSWER

Fewer kernel threads than processors. The kernel can schedule only that many of the program's threads at once. Some processors are left **idle** as far as this program is concerned, and the available parallelism is wasted. Also, if a kernel thread blocks (e.g. on I/O), even fewer user threads can run.

Q2.2 (5 pt) The number of kernel threads allocated to the program is equal to the number of processors. 2011

ANSWER

Equal to the number of processors. All processors can run the program's threads simultaneously, giving the best case while no thread blocks. However, when a kernel thread **blocks** (I/O, page fault, system call), its processor becomes **idle**. No spare kernel thread exists to run another ready user thread on it, so performance drops during blocking.

Q2.3 (5 pt) The number of kernel threads allocated to the program is greater than the number of processors but less than the number of userlevel threads. 2011

ANSWER

More kernel threads than processors, but fewer than user threads. This gives the best utilization. When one kernel thread blocks, the kernel can schedule **another ready kernel thread** (carrying another user thread) on that processor, so CPUs stay busy. The cost is some extra kernel overhead: more kernel threads to create, schedule and context-switch.

3. Multilevel queue scheduling

Q3.1 (2 pt) List two possible scheduling among queues, which we discussed in class. 2011

ANSWER

Two ways to schedule **among** the queues of a multilevel queue:

1. **Fixed-priority (preemptive) scheduling.** Serve all processes in a higher-priority queue (e.g. foreground/interactive) before any process in a lower queue (background/batch). A process arriving in a higher queue preempts a lower one. Lower queues may **starve**.
2. **Time slicing among queues.** Each queue gets a fixed share of CPU time, which it schedules among its own processes. Example: 80% to the foreground queue (RR) and 20% to the background queue (FCFS).

Q3.2 (8 pt) Describe how Linux support the above two possible inter-queue scheduling. 2011

ANSWER

Fixed priority between classes. Linux has two priority ranges mapped onto one global scale (lower number = higher priority). **Real-time** tasks are 0–99 and use POSIX SCHED_FIFO / SCHED_RR with static priorities.

Normal (time-sharing) tasks are 100–139, from nice −20...+19. A runnable real-time task always runs before any normal task, which is fixed-priority scheduling between the two "queues". With CFS, the scheduler likewise picks the highest-priority **scheduling class** first (real-time class before the default class).

Time slicing. In the O(1) scheduler (2.5) each task gets a time slice based on priority, and higher priority gets a longer quantum. Each CPU's runqueue has an **active** and an **expired** priority array. A task runs while it has slice left, then moves to expired. When the active array is empty, the arrays are **swapped**, so every task gets its share. In CFS (2.6.23+), instead of fixed quanta each task gets a **proportion** of CPU time within a **target latency**, weighted by nice value. The task with the smallest **virtual runtime (vruntime)** runs next. This is proportional sharing of time.

4. Consider a logical address space of 64 pages of 1024 words each, mapped onto a physical memory of 32 frames.

Q4.1 (5 pt) How many bits are there in the logical address? 2011

ANSWER

Pages: 64 = 2^6 gives a 6-bit page number. Page size: 1024 = 2^{10} words gives a 10-bit offset.
Logical address = 6 + 10 = 16 bits.

Q4.2 (5 pt) How many bits are there in the physical address? 2011

ANSWER

Frames: 32 = 2^5 gives a 5-bit frame number. The frame size equals the page size (2^{10}), so the offset is 10 bits.
Physical address = 5 + 10 = 15 bits.

5. A monitor `M` has processes `P1` and `P2` waiting in entry queue and process `P3` waiting in signal queue. Process `P0` is active in `M`. `M` also has two condition variables `x`, `y`. `x` has processes `P4` and `P5` waiting in its corresponding queue, and `y` has process `P6` waiting in its corresponding queue. Suppose `M` implements the signal and wait semantic. List the contents of each queue and the active process after the following events happen. (Note that each event happens right after the above state; that is, these events are independent to each other.)

Q5.1 (5 pt) New process `P7` accesses `M`. 2011

ANSWER

P7 just joins the end of the entry queue. Nothing else changes.

Active in M	Entry queue	Signal queue	x queue	y queue
P0	P1, P2, **P7**	P3	P4, P5	P6

Q5.2 (5 pt) `P0` executes `x.signal()`. 2011

ANSWER

Signal-and-wait (Hoare): the first waiter on x, **P4, resumes immediately**. The signaler **P0 is suspended in the signal queue**.

Active in M	Entry queue	Signal queue	x queue	y queue
P4	P1, P2	P3, **P0**	P5	P6

Q5.3 (5 pt) `P0` executes `y.wait()`. 2011

ANSWER

P0 joins y's queue and releases the monitor. The signal queue has priority over the entry queue, so **P3 becomes active**.

Active in M	Entry queue	Signal queue	x queue	y queue
P3	P1, P2	(empty)	P4, P5	P6, **P0**

Q6 (15 pt) Discuss how the following pairs of scheduling criteria conflict in certain settings. 2011

Qa) CPU utilization and response time 2011

ANSWER

CPU utilization vs. response time. CPU utilization is highest when context switches are rare, as with FCFS or a long quantum, because each switch is pure overhead. Good response time for interactive users needs **frequent preemption** (a short RR quantum) so every user gets the CPU quickly. That causes many context switches, so the CPU spends more time switching and less doing useful work, which lowers utilization.

Qb) Average turnaround time and maximum waiting time 2011

ANSWER

Average turnaround time vs. maximum waiting time. Average turnaround is minimized by running the **shortest jobs first** (SJF/SRTF). But a long job can be postponed again and again as short jobs keep arriving, so it may **starve**. Its waiting time, the **maximum** waiting time, becomes very large (unbounded). Bounding the maximum wait (e.g. with FCFS or aging) raises the average turnaround.

Qc) I/O device utilization and CPU utilization 2011

ANSWER

I/O device utilization vs. CPU utilization. Keeping devices busy means running **I/O-bound** processes as soon as their I/O completes, so they can issue the next request quickly. This means frequently preempting CPU-bound processes, and the context-switch overhead lowers CPU utilization. Maximizing CPU utilization means letting CPU-bound jobs run long without switching, which leaves the I/O devices **idle** while I/O-bound jobs wait for the CPU.

Qd) Deadline and fairness 2011

ANSWER

Deadline vs. fairness. To meet deadlines, the scheduler must always give the CPU to the most urgent process (earliest deadline or real-time priority), regardless of how much CPU it has already had. Other processes may wait a long time or **starve**, which is unfair. A **fair** scheduler gives everyone an equal share, so urgent tasks may not get enough CPU in time and **miss their deadlines**.

Qe) Throughput and context switch overhead 2011

ANSWER

Throughput vs. context-switch overhead. Throughput (jobs completed per unit time) is improved by finishing short jobs quickly, which needs preemption: SJF/SRTF or a small RR quantum. Every preemption adds **context-switch overhead**, during which no useful work is done. Minimizing switches (FCFS, large quantum) traps short jobs behind long ones (**convoy effect**), so fewer jobs complete per unit time. If the quantum is too small, overhead itself eats the throughput.

Q7 (20 pt) Consider the following Peterson's solution for the critical region problem: 2011

```
shared int turn;              /* set to i or j */
shared boolean flag[2];       /* set to T or F */

while (TRUE) {
    ...
    flag[i] = TRUE;
    turn = j;
    while (turn == j && flag[j] == TRUE) { /* no-op */ }
    ...
    /* critical region */
    ...
    flag[i] = FALSE;
    ...
    /* remainder region */
    ...
}
```

Prove Peterson's solution satisfy the three required critical region conditions.

ANSWER

Pi enters its critical region only when `flag[j] == FALSE` or `turn == i`.

1. Mutual exclusion. Suppose P0 and P1 were both in their critical regions. Then flag[0] = flag[1] = TRUE, so each must have passed the while because turn equalled its own index. That would require turn == 0 and turn == 1 at the same time, which is impossible. More precisely, turn is a single shared variable holding the value written **last**. The process that wrote turn last set it to the other process's index, so it must wait while the other is interested. Hence at most one is inside.

2. Progress. If Pj is not trying to enter (flag[j] == FALSE, i.e. it is in its remainder region), Pi passes the while at once. If both are trying, turn is either i or j, so exactly one of them passes, and the decision is made in finite time. Only processes in their entry sections take part; a process in its remainder section cannot block anyone.

3. Bounded waiting. Suppose Pi is waiting (turn == j and flag[j] == TRUE). When Pj leaves, it sets flag[j] = FALSE, so Pi can enter. If Pj immediately tries again, it sets turn = i, so Pj now waits and Pi enters. Pi therefore waits for at most **one** entry of Pj into its critical region.

Q8 (Bonus: 20 pt) Consider the code below as a variation of Peterson's solution above: 2011

```
shared int turn;              /* set to i or j */
shared boolean flag[2];       /* set to T or F */

while (TRUE) {
    ...
    flag[i] = TRUE;
    turn = i;     /* traditionally set to j */
    while (turn == j && flag[j] == TRUE) { /* no-op */ }
    ...
    /* critical region */
    ...
    flag[i] = FALSE;
    turn = j;     /* not usually done */
    ...
    /* remainder region */
    ...
}
```

Does the above solution satisfy the three required critical region conditions? If not, which property or properties is violated?

ANSWER

No. Mutual exclusion is violated. In the entry code each process now sets turn to **its own** index, so a process that has just written turn never waits (its test `turn == j` is false). Counter-example:

Step	P0	P1
1	flag[0] = TRUE; turn = 0	
2	test turn==1? false → **enters CR**	
3	(in critical region)	flag[1] = TRUE; turn = 1
4	(in critical region)	test turn==0? false → **enters CR**

Both are in the critical region. In general a newly arriving process always gets in, even while the other is inside. (Progress and bounded waiting still hold: a waiting Pi enters as soon as Pj exits, because the exit sets turn = i. But without mutual exclusion the solution is incorrect.)

2013 · CS5730 Operating Systems Midterm (2013/11/11)

1. Short Answer: Your answers should be as concise as possible; aim for three or four sentences.

Q1.1 (6 pt) Explain why implementing synchronization primitives by disabling interrupts is not appropriate in a single-processor system if the synchronization primitives are to be used in user-level programs. 2013

ANSWER

Disabling interrupts works on a uniprocessor only for **trusted kernel code**. It is the wrong tool for **user-level** synchronization because:

1. **It is a privileged instruction.** Letting user programs execute it breaks dual-mode protection; the hardware traps if user mode tries.
2. **The OS loses control of the CPU.** While interrupts are off, the **timer interrupt** cannot fire, so no preemption or context switch can happen. A buggy or malicious program could disable interrupts and never re-enable them (e.g. an infinite loop in its critical section). It would then **monopolize the CPU** and hang the whole system.
3. **Other events are delayed or lost.** I/O-completion interrupts and clock ticks are postponed, so devices sit idle, the system clock drifts and response time suffers. User critical sections can be long, which makes this worse.

(It also does not scale: on a multiprocessor it only stops the local CPU.) User programs should instead use OS-provided semaphores/mutexes or atomic hardware instructions such as test_and_set or swap.

Q1.2 (6 pt) What is busy-waiting in the discussion of critical section problem? Explain how busy-waiting may degrade the system performance, or even deadlock? 2013

ANSWER

Busy waiting (spinning) means a process waiting to enter its critical section repeatedly executes a loop that tests a condition, e.g. `while (test_and_set(&lock));` or `while (S <= 0);`. It never gives up the CPU while waiting.

Performance: the spinning process **wastes CPU cycles** that other processes could use. On a uniprocessor this is even worse: the spinner burns its whole time slice, and the lock holder cannot run during that time to finish and release the lock. So waiting gets longer and throughput drops.

Deadlock: with priority scheduling, suppose a high-priority process H spins on a lock held by a low-priority process L. H is always runnable, so the scheduler never runs L. L can never release the lock, and H spins forever. Both are stuck (a priority-inversion deadlock).

Fix: a blocking semaphore, where a waiting process sleeps on a queue instead of spinning.

Q1.3 (6 pt) In class, we discuss the bounded buffer problem which uses two semaphore, empty and full. Explain the usage of these two semaphores. 2013

ANSWER

Three semaphores protect a buffer of *n* slots: **mutex = 1** (mutual exclusion on the buffer), **empty = n** and **full = 0**.

- **empty** counts **empty slots**. The producer calls `wait(empty)` before inserting, so it blocks when the buffer is full. The consumer calls `signal(empty)` after removing an item.
- **full** counts **filled slots**. The consumer calls `wait(full)` before removing, so it blocks when the buffer is empty. The producer calls `signal(full)` after inserting.

```
Producer:                          Consumer:
do {                               do {
    /* produce item */                 wait(full);
    wait(empty);                       wait(mutex);
    wait(mutex);                       /* remove item */
    /* add item to buffer */           signal(mutex);
    signal(mutex);                     signal(empty);
    signal(full);                      /* consume item */
} while (true);                    } while (true);
```

Together they replace busy-wait loops, so all n slots can be used. Order matters: calling wait(mutex) before wait(empty) can deadlock.

Q1.4 (5 pt) Illustrate how the two process scheduling criteria, CPU utilization and throughput, may conflict with each other? 2013

ANSWER

CPU utilization measures how busy the CPU is; throughput measures how many processes **complete** per unit time. Maximizing one can hurt the other:

- Running **long CPU-bound jobs** keeps the CPU near 100% busy (high utilization). But each job takes a long time, so **few jobs finish** per unit time (low throughput). Short jobs stuck behind them also delay completion (convoy effect).
- Favoring **many short jobs** (SJF, short quantum) completes more jobs per unit time (high throughput). But it causes more context switches, whose overhead is not useful work, and short I/O-bound jobs leave the CPU idle while they wait on I/O. So utilization drops.

Q1.5 (6 pt) What happens if one thread of a multi-threaded process call `fork()`? What happens if one thread of a multi-threaded process call `exec()`? 2013

ANSWER

fork(): this depends on the system. Some UNIX systems provide **two versions** of fork(). One **duplicates all threads** of the process. The other (the POSIX behaviour) **duplicates only the thread that called fork()**, so the child is single-threaded. If exec() is called right after fork(), duplicating only the calling thread is enough, because exec replaces everything anyway. If the child does not call exec, duplicating all threads may be needed.

exec(): exec works as usual. The program named in its parameter **replaces the entire process, including all of its threads**. The other threads are destroyed, and the new program starts with a single thread.

Q2 (10 pt) A variation of the round-robin scheduler is the regressive round-robin scheduler. This scheduler assigns each process a time quantum and a priority. The initial value of a time quantum is 50 milliseconds. However, every time a process has been allocated the CPU and uses its entire time quantum (does not block for I/O), 10 milliseconds is added to its time quantum, and its priority level is boosted. (The time quantum for a process can be increased to a maximum of 100 milliseconds.) When a process blocks before using its entire time quantum, its time quantum is reduced by 5 milliseconds, but its priority remains the same. What type of process (CPU-bound or I/O-bound) does the regressive round-robin scheduler favor? Explain. 2013

ANSWER

It favors CPU-bound processes.

- A process that **uses its entire quantum** (does not block for I/O) is CPU-bound behaviour. It is **rewarded**: +10 ms of quantum, up to 100 ms, **and** a priority boost.
- A process that **blocks before using its quantum** is I/O-bound / interactive behaviour. It is **penalized**: −5 ms of quantum, and its priority is **not** raised.

So over time CPU-bound (batch) processes get longer time slices and higher priority, and I/O-bound processes get shorter slices and stay at lower priority. This is the opposite of the usual goal of favoring interactive processes.

Q3 (10 pt) Can a multi-threaded solution using multiple user-level threads achieve better performance on a multiprocessor system, which uses the many-to-one threading model, than on a single processor system? Explain. 2013

ANSWER

No, not significantly. In the many-to-one model, all user-level threads of the process are mapped onto **one kernel thread**. The kernel schedules only kernel threads, so it sees the process as a single schedulable entity. It can run on only **one processor at a time**, which means only one user thread executes at any moment even if other CPUs are idle. The program gets **no parallelism**, and its performance is essentially the same as on a single processor. Also, if any thread makes a blocking system call, the whole process (all threads) blocks.

(The only gain is indirect: other processes can use the other CPUs, so the program may face less competition.) To benefit from multiple processors you need one-to-one or many-to-many threading.

4. Multilevel queue scheduling

Q4.1 (4 pt) List two possible scheduling among queues, which we discussed in class. 2013

ANSWER

Two ways to schedule **among** the queues of a multilevel queue:

1. **Fixed-priority (preemptive) scheduling.** Serve all processes in a higher-priority queue (e.g. foreground/interactive) before any process in a lower queue (background/batch). A process arriving in a higher queue preempts a lower one. Lower queues may **starve**.
2. **Time slicing among queues.** Each queue gets a fixed share of CPU time, which it schedules among its own processes. Example: 80% to the foreground queue (RR) and 20% to the background queue (FCFS).

Q4.2 (5 pt) Why is it important for the scheduler to distinguish I/O-bound programs from CPU-bound programs? 2013

ANSWER

I/O-bound and CPU-bound processes behave very differently. **I/O-bound** processes have many short CPU bursts. **CPU-bound** processes have few, long bursts.

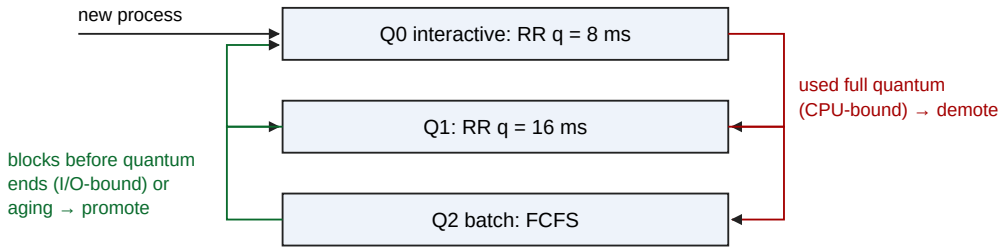
- If an I/O-bound process gets the CPU quickly, it issues its next I/O request soon and then gets out of the way. The **devices stay busy** while CPU-bound processes use the CPU, so **both CPU and I/O utilization** improve.
- I/O-bound processes are usually **interactive**, so serving them first gives good **response time**.
- If they wait behind long CPU bursts (convoy effect), devices go idle and users see poor response.

So the scheduler should give I/O-bound processes higher priority and short quanta, and give CPU-bound processes lower priority with longer quanta (fewer switches).

Q4.3 (6 pt) Modify multilevel queue scheduling so it can distinguish I/O-bound programs from CPU-bound programs. 2013

ANSWER

Change it into a **multilevel feedback queue**, where a process can **move between queues** based on its observed CPU bursts:



- A new process enters **Q0** (small quantum). If it **uses the full quantum**, it is behaving CPU-bound and is **demoted** to Q1, then to Q2.
- If it **blocks for I/O before its quantum expires**, it is I/O-bound and stays in, or is **promoted** to, a higher queue.
- **Aging:** a process that waits too long in a low queue is moved up, which prevents starvation.

So I/O-bound processes end up in high-priority, short-quantum queues and CPU-bound processes in low-priority, long-quantum queues. Parameters: number of queues, algorithm per queue, promotion rule, demotion rule, entry queue.

5.

Q5.1 (5 pt) Write to demonstrate how to use the atomic instruction TestAndSet for critical section problem. 2013

ANSWER

```
boolean test_and_set(boolean *target) {   /* executed ATOMICALLY by hardware */
    boolean rv = *target;
    *target = true;
    return rv;
}

boolean lock = false;                     /* shared */

do {
    while (test_and_set(&lock))
        ;                                 /* spin while lock was already true */
    /* critical section */
    lock = false;                         /* release */
    /* remainder section */
} while (true);
```

The first process to call test_and_set gets back **false** (the lock was free) and has atomically set lock to true, so it enters. Every other caller gets **true** and keeps looping until the holder sets lock = false. This gives mutual exclusion. (Bounded waiting needs an extra `waiting[]` array.)

Q5.2 (5 pt) Illustrate how your solution will fail if TestAndSet is not atomic. 2013

ANSWER

If test_and_set is not atomic, its read (`rv = *target`) and write (`*target = true`) can be interleaved. Start with lock = false:

Step	P0	P1	lock
1	rv = lock → false		false
2	*preempted*	rv = lock → false	false
3		lock = true; returns false → **enters CS**	true
4	lock = true; returns false → **enters CS**		true

Both processes saw the lock as free, so **both are in the critical section: mutual exclusion is violated.**

Q5.3 (5 pt) Write to demonstrate how to use the atomic instruction Swap for critical section problem. 2013

ANSWER

```
/* swap: executed ATOMICALLY by hardware */
void swap(boolean *a, boolean *b) {
    boolean temp = *a;
    *a = *b;
    *b = temp;
}

boolean lock = false;           /* shared */
boolean key;                    /* local to each process */

do {
    key = true;
    while (key == true)
        swap(&lock, &key);  /* key=old lock; lock=true */
    /* critical section */
    lock = false;
    /* remainder section */
} while (true);
```

A process loops until a swap returns **key = false**, meaning the lock was free. That same swap atomically set lock = true, so only one process gets out of the loop. The others keep getting key = true until lock is reset to false.

Q5.4 (5 pt) Illustrate how your solution will fail if Swap is not atomic. 2013

ANSWER

If swap is not atomic, the three steps (`temp=*a; *a=*b; *b=temp`) of two processes can interleave. Start with lock = false, and each process has key = true:

Step	P0	P1	lock
1	temp0 = lock → false		false
2	*preempted*	temp1 = lock → false	false
3		lock = key1 (true); key1 = temp1 = false → **enters CS**	true
4	lock = key0 (true); key0 = temp0 = false → **enters CS**		true

Both read the old value false before either wrote true, so both leave the loop. **Mutual exclusion is violated.**

Q6 (10 pt) Race conditions are possible in many computer systems. Consider a banking system with two methods: `deposit(amount)` and `withdraw(amount)`. These two methods are passed the amount that is to be deposited or withdrawn from a bank account. Assume that a husband and wife share a bank account and that concurrently the husband calls the `withdraw()` method and the wife calls `deposit()`. 2013

(The question stops here on the paper. The textbook version asks: describe how a race condition is possible and what might be done to prevent it.)

ANSWER

deposit() does `balance = balance + amount` and withdraw() does `balance = balance - amount`. At machine level each is **load balance → modify in a register → store balance**, which is not atomic. Suppose balance = 100, the husband withdraws 50 and the wife deposits 50 (the correct result is 100):

Step	Husband: withdraw(50)	Wife: deposit(50)	balance
1	reg1 = balance (100)		100
2		reg2 = balance (100)	100
3	reg1 = 100 − 50; balance = 50		50
4		reg2 = 100 + 50; balance = 150	**150**

The withdrawal is lost, and the result would be 50 if steps 3 and 4 were swapped. The outcome depends on the order of execution, which is a **race condition**.

Prevention: make the update a critical section, so only one method can touch balance at a time. Use a lock or semaphore (`wait(mutex); balance ±= amount; signal(mutex);`), a monitor, or synchronized methods.

Q7 (6 pt) Consider the following code segment: 2013

```
pid_t pid;

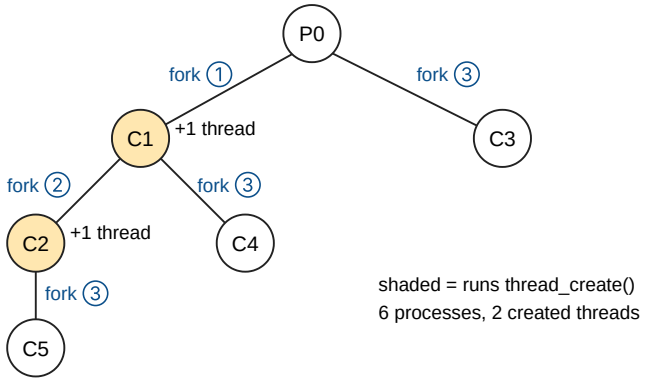
pid = fork();
if (pid == 0) { /* child process */
    fork();
    thread_create( . . .);
}
fork();
```

a) How many unique processes are created?
b) How many unique threads are created?

ANSWER

Trace each fork (the original process is P0):

- fork ① gives P0 and child C1. Only C1 has pid == 0.
- Inside the if, C1 calls fork ②, creating C2. **C1 and C2 each call thread_create()**, so 2 threads are created.
- The final fork ③ is executed by P0, C1 and C2, creating C3, C4 and C5.



> **(a) 6 unique processes** (P0, C1–C5; 5 newly created if the original is not counted).
> **(b) 2 unique threads** created by thread_create(), one in C1 and one in C2. The last fork() duplicates only the calling thread (POSIX), so it creates no extra threads.

2019 · CS5730 Operating Systems Midterm (2019/12/12)

1. Short Answer: Your answers should be as concise as possible; aim for three or four sentences.

Q1.1 (5 pt) Ordinarily the `exec()` system call follows the `fork()`. Explain what would happen if a programmer were to inadvertently place the call to `exec()` before the call to `fork()`. 2019

ANSWER

exec() **replaces the calling process's entire memory image** (code, data, heap, stack) with the program named in its parameters. On success, **it never returns**.

So if exec() is placed before fork(), the new program starts running in place of the original one, under the **same process ID**. The rest of the original code, **including the fork() call, is gone and never executed**. **No child process is created**, and the original program's remaining work is not done.

Only if exec() **fails** (returns −1) does execution continue to the fork().

Q1.2 (5 pt) What happens if one thread of a multi-threaded process call `fork()`? What happens if one thread of a multi-threaded process call `exec()`? 2019

ANSWER

fork(): this depends on the system. Some UNIX systems provide **two versions** of fork(). One **duplicates all threads** of the process. The other (the POSIX behaviour) **duplicates only the thread that called fork()**, so the child is single-threaded. If exec() is called right after fork(), duplicating only the calling thread is enough, because exec replaces everything anyway. If the child does not call exec, duplicating all threads may be needed.

exec(): exec works as usual. The program named in its parameter **replaces the entire process, including all of its threads**. The other threads are destroyed, and the new program starts with a single thread.

Q1.3 (5 pt) What is processor affinity? Explain why it needed to be considered during process scheduling on multiple CPU systems. 2019

ANSWER

Processor affinity: a process has an affinity for the processor it is currently running on. It is kept on (or restricted to) that same CPU rather than migrated to another one.

Why it matters on multiple-CPU systems: while a process runs, its recently used data fills that processor's **cache**. If the scheduler migrates it to another CPU, the old cache contents must be **invalidated** and the new processor's cache must be **repopulated**. That causes many cache misses and memory stalls, which is expensive. Keeping the process on the same CPU reuses the "warm" cache. On NUMA systems it also keeps the process close to its local memory.

Soft affinity: the OS tries to keep the process on the same CPU but may migrate it. **Hard affinity:** the process specifies the set of CPUs it may run on. The scheduler must balance affinity against **load balancing** (push/pull migration), which moves processes from busy to idle CPUs.

Q2 (10 pt) One technique for implementing lottery scheduling works by assigning processes lottery tickets, which are used for allocating CPU time. Whenever a scheduling decision has to be made, a lottery ticket is chosen at random, and the process holding that ticket gets the CPU. The BTV operating system implements lottery scheduling by holding a lottery 50 times each second, with each lottery winner getting 20 milliseconds of CPU time (20 milliseconds × 50 = 1 second). Describe how the BTV scheduler can ensure that higher-priority threads receive more attention from the CPU than lower-priority threads. 2019

ANSWER

Give **higher-priority threads more lottery tickets**. At each of the 50 drawings per second, the winning ticket is chosen uniformly at random. A thread holding *k* of the *T* tickets therefore wins with probability k/T, and on average receives about **k/T of the CPU time** (k/T × 50 × 20 ms per second).

Example: high-priority thread H has 30 tickets and low-priority L has 10. H wins about 75% of the drawings (≈ 750 ms per second) and L about 25% (≈ 250 ms). The scheduler can also adjust priority dynamically by giving or taking away tickets, e.g. extra tickets to a thread that has waited long. Every thread with at least one ticket eventually wins, so there is no starvation.

Q3 (15 pt) A variation of the round-robin scheduler is the **regressive round-robin scheduler**. This scheduler assigns each process a time quantum and a priority. The initial value of a time quantum is 50 milliseconds. However, every time a process has been allocated the CPU and uses its entire time quantum (does not block for I/O), 10 milliseconds is added to its time quantum, and its priority level is boosted. (The time quantum for a process can be increased to a maximum of 100 milliseconds.) When a process blocks before using its entire time quantum, its time quantum is reduced by 5 milliseconds, but its priority remains the same. What type of process, namely CPU-bound (such as batch processes) or I/O-bound (such as interactive processes), does the regressive round-robin scheduler favor? Explain. 2019

ANSWER

It favors CPU-bound processes.

- A process that **uses its entire quantum** (does not block for I/O) is CPU-bound behaviour. It is **rewarded**: +10 ms of quantum, up to 100 ms, **and** a priority boost.
- A process that **blocks before using its quantum** is I/O-bound / interactive behaviour. It is **penalized**: −5 ms of quantum, and its priority is **not** raised.

So over time CPU-bound (batch) processes get longer time slices and higher priority, and I/O-bound processes get shorter slices and stay at lower priority. This is the opposite of the usual goal of favoring interactive processes.

Q4 (10 pt) Consider the following code segment: 2019

```
pid_t pid;

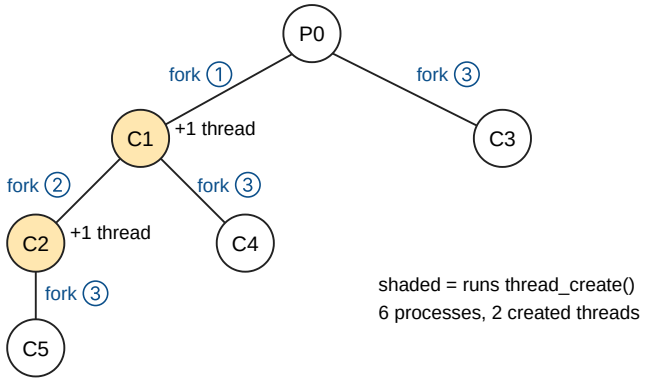
pid = fork();
if (pid == 0) { /* child process */
    fork();
    thread_create( . . .);
}
fork();
```

a) How many unique processes are created?
b) How many unique threads are created?

ANSWER

Trace each fork (the original process is P0):

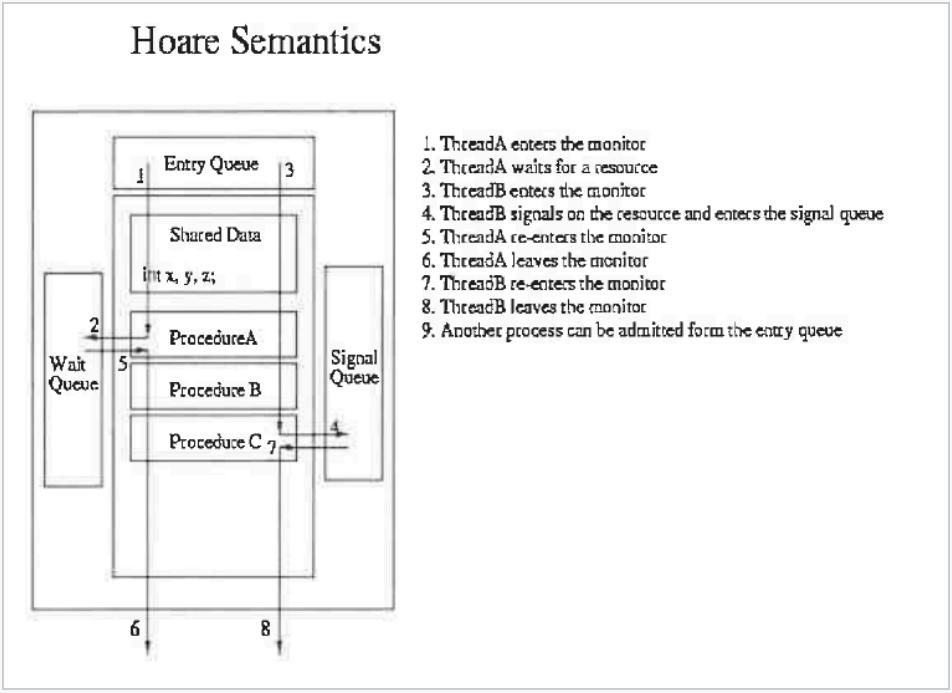
- fork ① gives P0 and child C1. Only C1 has pid == 0.
- Inside the if, C1 calls fork ②, creating C2. **C1 and C2 each call thread_create()**, so 2 threads are created.
- The final fork ③ is executed by P0, C1 and C2, creating C3, C4 and C5.



(a) 6 unique processes (P0, C1–C5; 5 newly created if the original is not counted).
(b) 2 unique threads created by thread_create(), one in C1 and one in C2. The last fork() duplicates only the calling thread (POSIX), so it creates no extra threads.

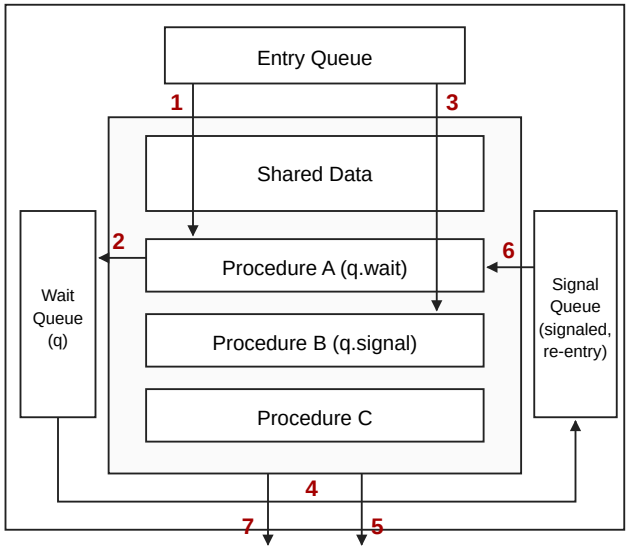
2019

Q5 (10 pt) Note that only one process is active within the monitor. Suppose `A` associated with the condition variable `q` is resumed when `B` invokes `q.signal()`. We discussed two possibile solutions in class. One is **signal and wait**, and the flow is illustrated in the following (Hoare semantics). Please modify the Hoare semantics and depict the other solution (i.e., **signal and continue**).



ANSWER

Signal-and-continue: when B executes `q.signal()`, **B keeps running** in the monitor. The waiting process A is only moved from q's wait queue to a ready/re-entry queue (the "signal queue" now holds the *signaled* process). A resumes after B leaves the monitor or waits.



1. Thread A enters the monitor
2. A waits on q → q's wait queue (monitor released)
3. Thread B enters the monitor
4. B signals q: A is moved from wait queue → signal queue; **B CONTINUES running**
5. B leaves the monitor
6. A re-enters the monitor (signal queue has priority over entry queue)
7. A leaves the monitor
8. Another process can be admitted from the entry queue

Difference from Hoare: signal queue now holds the SIGNALED process, not the signaler; 8 steps instead of 9.

Compared with Hoare: in Hoare's version A runs immediately and the signaler B waits in the signal queue. In signal-and-continue, A may find the condition changed by the time it runs, so waiters should re-check it with `while (!cond) q.wait();`. There are fewer context switches, and this is what Java and Pthreads use.

Q6 (20 pt) In class, we have illustrated the solution for dining-philosopher problem using monitor. 2019
However, as I mentioned, the solution is conceptual only since there is no monitor mechanism. We also discussed how to implement a monitor mechanism with **signal and wait** via semaphores, and thus the dining-philosopher problem solution together with the monitor mechanism with **signal and wait** via semaphores is a complete solution. Please modify the monitor mechanism with **signal and wait** via semaphores to **signal and continue** and integrate them into dining-philosopher problem solution. That is, I would like you to present a complete code for dining-philosopher problem solution which uses a monitor mechanism with **signal and continue** via semaphores.

ANSWER

Combine the dining-philosopher monitor with the signal-and-continue implementation above. `self[i]` becomes semaphore `self_sem[i]` plus counter `self_count[i]`.

```
enum {THINKING, HUNGRY, EATING} state[5];
semaphore mutex = 1;              // monitor lock
semaphore next  = 0;      // signaled, waiting to re-enter
int next_count  = 0;
semaphore self_sem[5]   = {0,0,0,0,0};  // cond. self[i]
int       self_count[5] = {0,0,0,0,0};

void leave() {                // release the monitor
    if (next_count > 0) signal(next);
    else                signal(mutex);
}
void self_wait(int i) {          // self[i].wait()
    self_count[i]++;
    leave();                  // release monitor
    wait(self_sem[i]);           // sleep until signaled
    wait(next);               // re-enter after signaler
    next_count--;
}
void self_signal(int i) {     // self[i].signal(), S&C
    if (self_count[i] > 0) {
        self_count[i]--;
        next_count++;
        signal(self_sem[i]);  // wake i; caller continues
    }
```

```
}
void test(int i) {            // only inside the monitor
    if (state[(i+4)%5] != EATING && state[i] == HUNGRY &&
        state[(i+1)%5] != EATING) {
        state[i] = EATING;
        self_signal(i);
    }
}
void pickup(int i) {
    wait(mutex);                // enter monitor
    state[i] = HUNGRY;
    test(i);
    while (state[i] != EATING)  // re-check after waking
        self_wait(i);
    leave();
}
void putdown(int i) {
    wait(mutex);                // enter monitor
    state[i] = THINKING;
    test((i+4)%5);            // left neighbor may eat
    test((i+1)%5);            // right neighbor may eat
    leave();
}
void init() {
    for (int i = 0; i < 5; i++) state[i] = THINKING;
}

void philosopher(int i) {
    while (true) {
        think(); pickup(i); eat(); putdown(i);
    }
}
```

It is correct under signal-and-continue because test() sets state[i] = EATING **before** signaling, and only philosopher i can change it back. So when i resumes, its condition still holds. A philosopher eats only when neither neighbor eats, so there is no deadlock (starvation is still possible).

7. Multilevel queue scheduling

Q7.1 (5 pt) List two possible scheduling among queues, which we discussed in class. 2019

ANSWER

Two ways to schedule **among** the queues of a multilevel queue:

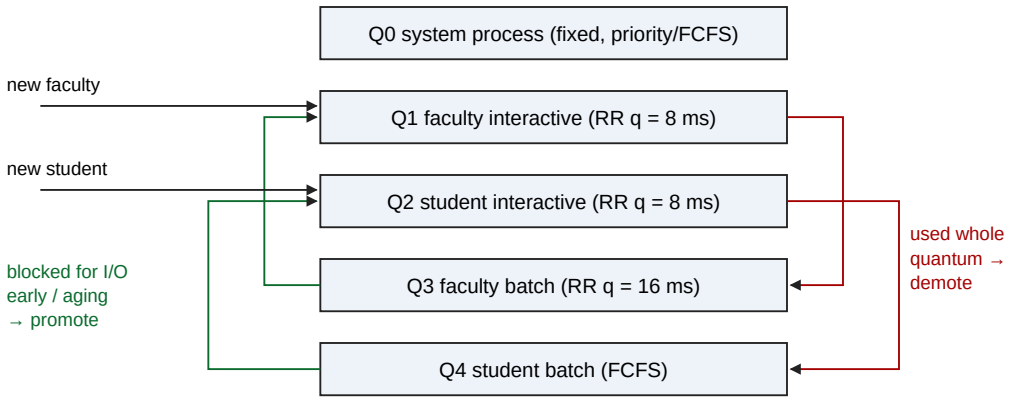
1. **Fixed-priority (preemptive) scheduling.** Serve all processes in a higher-priority queue (e.g. foreground/interactive) before any process in a lower queue (background/batch). A process arriving in a higher queue preempts a lower one. Lower queues may **starve**.
2. **Time slicing among queues.** Each queue gets a fixed share of CPU time, which it schedules among its own processes. Example: 80% to the foreground queue (RR) and 20% to the background queue (FCFS).

Q7.2 (5 pt) Modify the following multilevel queue scheduling system such that a new process will automatically enter the appropriate queue after a few runs. 2019

system process
faculty interactive process
student interactive process
faculty batch process
student batch process

ANSWER

Turn it into a **multilevel feedback queue**: a process may **move between queues** according to its observed behaviour, so it reaches the right queue automatically after a few runs.



Higher queue preempts lower (fixed priority between queues); Q0 never moves.

- A new process enters the **interactive** queue of its class (faculty → Q1, student → Q2), which uses a short quantum.
- If it **uses its whole quantum**, it is CPU-bound, so it is **demoted** to the batch queue of its class (Q1→Q3, Q2→Q4).
- If it **blocks for I/O before the quantum ends**, it is interactive, so it stays in or is **promoted** back to the interactive queue (Q3→Q1, Q4→Q2).
- **Aging** promotes processes that wait too long, preventing starvation. System processes stay fixed in Q0.

Defined by: number of queues, scheduling algorithm per queue, when to promote, when to demote, and which queue a new process enters.

Q8 (10 pt) A student is asked to write a program which will create a hundred POSIX threads. Each thread will print out the value of the parameter passed from `main()` which are 1, 2, 3, up to 100 respectively. Consider the following code fragment he wrote, please verify its correctness. If incorrect, please correct the error. 2019

```
void *doSomething(void *y) {
    ...
    printf("%d\n", *y);
    ...
}

int main() {
    pthread_t t1[100];          /* declare a thread */
    int param1, i;

    param1 = 1;
    for (i = 0; i < 100; i++) {
        param1++;
        pthread_create(&(t1[i]), NULL, doSomething, &param1);
    }
    ...
}
```

ANSWER

Incorrect. Problems:

1. **All 100 threads receive the same address** `&param1`**.** main keeps incrementing param1 while the threads run, so each thread prints whatever value param1 holds when that thread happens to run. The output has duplicates and missing numbers (often many 100s), not 1...100.
2. main never calls **pthread_join**, so it may return (ending the process and all its threads) before the threads print.
3. `*y` dereferences a `void *`, so it needs a cast to `int *`.

Corrected: give every thread its own argument and join all threads.

```
void *doSomething(void *y) {
    ...
    printf("%d\n", *(int *)y);
    ...
    pthread_exit(0);
}

int main() {
    pthread_t t1[100];            /* declare threads */
    int param[100], i;      /* one argument per thread */

    for (i = 0; i < 100; i++) {
        param[i] = i + 1;         /* 1, 2, ..., 100 */
        pthread_create(&(t1[i]), NULL, doSomething, &param[i]);
    }
    for (i = 0; i < 100; i++)
        pthread_join(t1[i], NULL);  /* wait for all */
    return 0;
}
```

(If the numbers must also be printed *in order*, the threads must additionally be synchronized, e.g. by joining each thread right after creating it.)

2020 · CS5730 Operating Systems (2020/12/1; the paper is titled "Final")

Only the photographed pages are available (Q1–Q8).

1. Short Answer: Your answers should be as concise as possible; aim for three or four sentences.

Q1.1 (5 pt) Ordinarily the `exec()` system call follows the `fork()`. Explain what would happen if a programmer were to inadvertently place the call to `exec()` before the call to `fork()`. 2020

ANSWER

exec() **replaces the calling process's entire memory image** (code, data, heap, stack) with the program named in its parameters. On success, **it never returns**.

So if exec() is placed before fork(), the new program starts running in place of the original one, under the **same process ID**. The rest of the original code, **including the fork() call, is gone and never executed**. **No child process is created**, and the original program's remaining work is not done.

Only if exec() **fails** (returns −1) does execution continue to the fork().

Q1.2 (5 pt) What happens if one thread of a multi-threaded process call `fork()`? What happens if one thread of a multi-threaded process call `exec()`? 2020

ANSWER

fork(): this depends on the system. Some UNIX systems provide **two versions** of fork(). One **duplicates all threads** of the process. The other (the POSIX behaviour) **duplicates only the thread that called fork()**, so the child is single-threaded. If exec() is called right after fork(), duplicating only the calling thread is enough, because exec replaces everything anyway. If the child does not call exec, duplicating all threads may be needed.

exec(): exec works as usual. The program named in its parameter **replaces the entire process, including all of its threads**. The other threads are destroyed, and the new program starts with a single thread.

Q1.3 (5 pt) What is processor affinity? Explain why it needed to be considered during process scheduling on multiple CPU systems. 2020

ANSWER

Processor affinity: a process has an affinity for the processor it is currently running on. It is kept on (or restricted to) that same CPU rather than migrated to another one.

Why it matters on multiple-CPU systems: while a process runs, its recently used data fills that processor's **cache**. If the scheduler migrates it to another CPU, the old cache contents must be **invalidated** and the new processor's cache must be **repopulated**. That causes many cache misses and memory stalls, which is expensive. Keeping the process on the same CPU reuses the "warm" cache. On NUMA systems it also keeps the process close to its local memory.

Soft affinity: the OS tries to keep the process on the same CPU but may migrate it. **Hard affinity:** the process specifies the set of CPUs it may run on. The scheduler must balance affinity against **load balancing** (push/pull migration), which moves processes from busy to idle CPUs.

Q2 (10 pt) One technique for implementing lottery scheduling works by assigning processes lottery tickets, which are used for allocating CPU time. Whenever a scheduling decision has to be made, a lottery ticket is chosen at random, and the process holding that ticket gets the CPU. The BTV operating system implements lottery scheduling by holding a lottery 50 times each second, with each lottery winner getting 20 milliseconds of CPU time (20 milliseconds × 50 = 1 second). Describe how the BTV scheduler can ensure that higher-priority threads receive more attention from the CPU than lower-priority threads. 2020

ANSWER

Give **higher-priority threads more lottery tickets**. At each of the 50 drawings per second, the winning ticket is chosen uniformly at random. A thread holding *k* of the *T* tickets therefore wins with probability k/T, and on average receives about **k/T of the CPU time** (k/T × 50 × 20 ms per second).

Example: high-priority thread H has 30 tickets and low-priority L has 10. H wins about 75% of the drawings (≈ 750 ms per second) and L about 25% (≈ 250 ms). The scheduler can also adjust priority dynamically by giving or taking away tickets, e.g. extra tickets to a thread that has waited long. Every thread with at least one ticket eventually wins, so there is no starvation.

Q3 (10 pt) A variation of the round-robin scheduler is the **regressive round-robin scheduler**. This scheduler assigns each process a time quantum and a priority. The initial value of a time quantum is 50 milliseconds. However, every time a process has been allocated the CPU and uses its entire time quantum (does not block for I/O), 10 milliseconds is added to its time quantum, and its priority level is boosted. (The time quantum for a process can be increased to a maximum of 100 milliseconds.) When a process blocks before using its entire time quantum, its time quantum is reduced by 5 milliseconds, but its priority remains the same. What type of process, namely CPU-bound (such as batch processes) or I/O-bound (such as interactive processes), does the regressive round-robin scheduler favor? Explain. 2020

ANSWER

It favors CPU-bound processes.

- A process that **uses its entire quantum** (does not block for I/O) is CPU-bound behaviour. It is **rewarded**: +10 ms of quantum, up to 100 ms, **and** a priority boost.
- A process that **blocks before using its quantum** is I/O-bound / interactive behaviour. It is **penalized**: −5 ms of quantum, and its priority is **not** raised.

So over time CPU-bound (batch) processes get longer time slices and higher priority, and I/O-bound processes get shorter slices and stay at lower priority. This is the opposite of the usual goal of favoring interactive processes.

Q4 (10 pt) Consider the following code segment: 2020

```
pid_t pid;

pid = fork();
if (pid == 0) { /* child process */
    fork();
    thread_create( . . .);
}
fork();
```

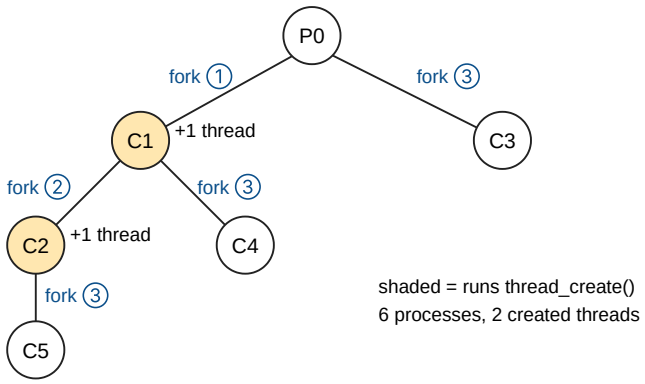
a) How many unique processes are created?
b) How many unique threads are created?

ANSWER

Trace each fork (the original process is P0):

- fork ① gives P0 and child C1. Only C1 has pid == 0.
- Inside the if, C1 calls fork ②, creating C2. **C1 and C2 each call thread_create()**, so 2 threads are created.

- The final fork ③ is executed by P0, C1 and C2, creating C3, C4 and C5.



(a) 6 unique processes (P0, C1–C5; 5 newly created if the original is not counted).
(b) 2 unique threads created by thread_create(), one in C1 and one in C2. The last fork() duplicates only the calling thread (POSIX), so it creates no extra threads.

Q5 (15 pt) Discuss how the following pairs of scheduling criteria conflict in certain settings. 2020

Q5.1 CPU utilization and response time 2020

ANSWER

CPU utilization vs. response time. CPU utilization is highest when context switches are rare, as with FCFS or a long quantum, because each switch is pure overhead. Good response time for interactive users needs **frequent preemption** (a short RR quantum) so every user gets the CPU quickly. That causes many context switches, so the CPU spends more time switching and less doing useful work, which lowers utilization.

Q5.2 Average turnaround time and maximum waiting time 2020

ANSWER

Average turnaround time vs. maximum waiting time. Average turnaround is minimized by running the **shortest jobs first** (SJF/SRTF). But a long job can be postponed again and again as short jobs keep arriving, so it may **starve**. Its waiting time, the **maximum** waiting time, becomes very large (unbounded). Bounding the maximum wait (e.g. with FCFS or aging) raises the average turnaround.

Q5.3 I/O device utilization and CPU utilization 2020

ANSWER

I/O device utilization vs. CPU utilization. Keeping devices busy means running **I/O-bound** processes as soon as their I/O completes, so they can issue the next request quickly. This means frequently preempting CPU-bound processes, and the context-switch overhead lowers CPU utilization. Maximizing CPU utilization means letting CPU-bound jobs run long without switching, which leaves the I/O devices **idle** while I/O-bound jobs wait for the CPU.

Q5.4 Deadline and fairness 2020

ANSWER

Deadline vs. fairness. To meet deadlines, the scheduler must always give the CPU to the most urgent process (earliest deadline or real-time priority), regardless of how much CPU it has already had. Other processes may wait a long time or **starve**, which is unfair. A **fair** scheduler gives everyone an equal share, so urgent tasks may not get enough CPU in time and **miss their deadlines**.

Q5.5 Throughput and context switch overhead 2020

ANSWER

Throughput vs. context-switch overhead. Throughput (jobs completed per unit time) is improved by finishing short jobs quickly, which needs preemption: SJF/SRTF or a small RR quantum. Every preemption adds **context-switch overhead**, during which no useful work is done. Minimizing switches (FCFS, large quantum) traps short jobs behind long ones (**convoy effect**), so fewer jobs complete per unit time. If the quantum is too small, overhead itself eats the throughput.

Q6 (10 pt) A student is asked to write a program which will create a hundred POSIX threads. Each thread will print out the value of the parameter passed from `main()` which are 1, 2, 3, up to 100 respectively. Consider the following code fragment he wrote, please verify its correctness. If incorrect, please correct the error. 2020

```
void *doSomething(void *y) {
    ...
    printf("%d\n", *y);
    ...
}

int main() {
    pthread_t t1[100];          /* declare a thread */
    int param1, i;

    param1 = 1;
    for (i = 0; i < 100; i++) {
        param1++;
        pthread_create(&(t1[i]), NULL, doSomething, &param1);
    }
    ...
}
```

ANSWER

Incorrect. Problems:

1. **All 100 threads receive the same address** `&param1`**.** main keeps incrementing param1 while the threads run, so each thread prints whatever value param1 holds when that thread happens to run. The output has duplicates and missing numbers (often many 100s), not 1…100.
2. main never calls **pthread_join**, so it may return (ending the process and all its threads) before the threads print.
3. `*y` dereferences a `void *`, so it needs a cast to `int *`.

Corrected: give every thread its own argument and join all threads.

```
void *doSomething(void *y) {
    ...
    printf("%d\n", *(int *)y);
    ...
    pthread_exit(0);
}

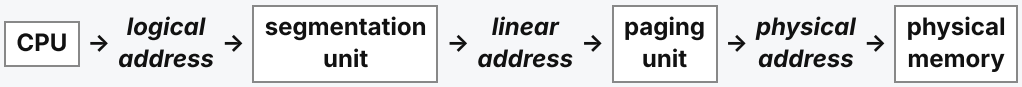
int main() {
    pthread_t t1[100];            /* declare threads */
    int param[100], i;      /* one argument per thread */

    for (i = 0; i < 100; i++) {
        param[i] = i + 1;         /* 1, 2, ..., 100 */
        pthread_create(&(t1[i]), NULL, doSomething, &param[i]);
    }
    for (i = 0; i < 100; i++)
```

```
        pthread_join(t1[i], NULL);  /* wait for all */
    return 0;
}
```

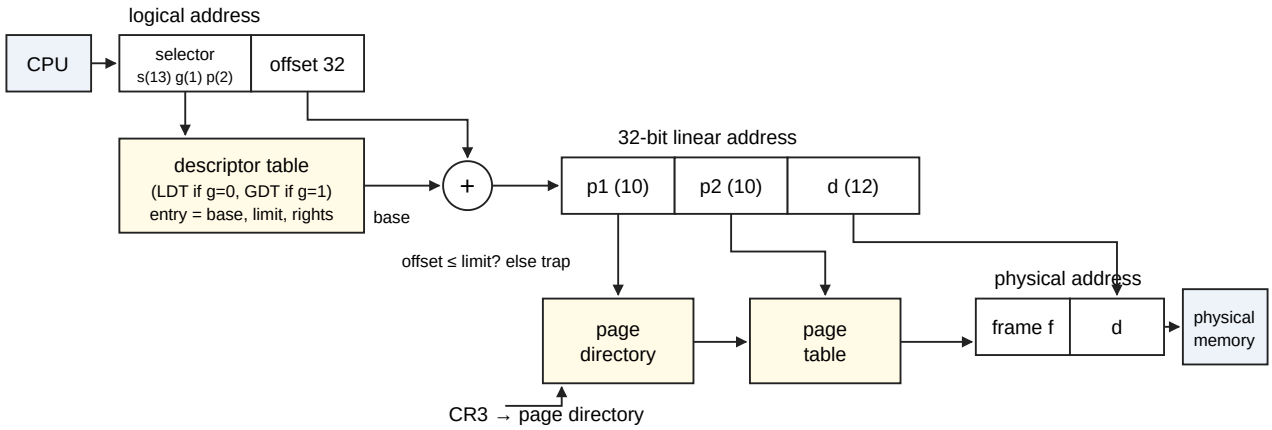
(If the numbers must also be printed *in order*, the threads must additionally be synchronized, e.g. by joining each thread right after creating it.)

7. Consider the Intel address translation scheme shown in the following.



Q7.1 (10 pt) Illustrate all the steps that the Intel 80386 takes in translating a logical address into a physical address. 2020

ANSWER



1. The CPU generates a **logical address = (selector, offset)**. The 16-bit selector has **s** = 13-bit segment number, **g** = 1 bit choosing GDT or LDT, and **p** = 2 protection bits.
2. The **segmentation unit** uses s and g to fetch the 8-byte **segment descriptor** from the **LDT** (private segments) or **GDT** (shared segments). Descriptors are cached in segment registers. It checks the offset against the segment **limit** and the access rights, trapping on a violation, then computes the **linear address = base + offset** (32 bits).
3. The **paging unit** splits the linear address into **p1 (10 bits) | p2 (10 bits) | d (12 bits)**. Register **CR3** points to the current **page directory**. p1 selects a directory entry, which points to a **page table**. p2 selects the page-table entry, which gives **frame f**.
4. **Physical address = f concatenated with d**, which is sent to memory. (For 4 MB pages the directory entry points straight at the frame, with a 22-bit offset. The TLB caches recent translations.)

Q7.2 (5 pt) What are the advantages to the operating system of hardware that provides such complicated memory translation hardware? 2020

ANSWER

- **Segmentation** matches the programmer's view (code, data and stack as separate logical units). It gives **per-segment protection** (e.g. read-only, execute-only code, privilege levels) and easy **sharing** of whole segments between processes.
- **Paging** on top removes **external fragmentation**. Segments need not be contiguous in physical memory; they are allocated in fixed-size frames, so allocation is simple.
- The OS gets **flexibility**: pages can be swapped or loaded on demand (virtual memory), shared at page level, and protected with page bits. The hardware does the work at memory speed, so the OS does not translate in software.

Q7.3 (5 pt) Are there any disadvantages to this address-translation system? If so, what are they? If not, why is it not used by every manufacturer? 2020

ANSWER

Yes.

- **Slower access:** without TLB hits, each reference needs a segment-descriptor lookup plus **two page-table memory accesses** (directory and table) before the data access itself.
- **Complexity and cost:** more hardware (segment registers, descriptor tables, MMU, TLB), more chip area and power. The OS must manage both segment tables and page tables.
- **Memory overhead** for the tables, and paging still has internal fragmentation.

Because of this cost and complexity, many manufacturers (most RISC designs, ARM, and x86-64 in practice with a flat segment model) use **paging only**. It delivers most of the benefit with simpler, faster hardware.

Q8 (10 pt) Assume we have a demand-paged memory. The page table is held in registers. It takes 8 milliseconds to service a page fault if an empty page is available or the replaced page is not modified, and 20 milliseconds if the replaced page is modified. Memory access time is 100 nanoseconds. Assume that the page to be replaced is modified 70 percent of the time. What is the maximum acceptable page-fault rate for an effective access time of no more than 200 nanoseconds? 2020

ANSWER

Let p = page-fault rate. Memory access = 100 ns. A page-fault service averages:

0.3 × 8 ms + 0.7 × 20 ms = 2.4 + 14 = **16.4 ms = 16,400,000 ns**

Effective access time: EAT = (1 − p) × 100 + p × 16,400,000 ≤ 200

100 + 16,399,900 p ≤ 200 → p ≤ 100 / 16,399,900

$p \leq 6.1 \times 10^{-6}$ (about 1 fault per 164,000 memory accesses).

Keep units consistent (ms → ns). Since the page table is in registers, a non-faulting access costs just one memory access (100 ns).

2021 · CS5730 Operating Systems Midterm (2021/11/30)

Both 2021 PDFs ("Operating System Exam_2021" and "final-2021f") contain this same exam.

Essay Questions: Your answers should be as concise and clear as possible

Q1 (5 pt) Ordinarily the `exec()` system call follows the `fork()`. Explain what would happen if a programmer were to inadvertently place the call to `exec()` before the call to `fork()`. 2021

ANSWER

exec() **replaces the calling process's entire memory image** (code, data, heap, stack) with the program named in its parameters. On success, **it never returns**.

So if exec() is placed before fork(), the new program starts running in place of the original one, under the **same process ID**. The rest of the original code, **including the fork() call, is gone and never executed**. **No child process is created**, and the original program's remaining work is not done.

Only if exec() **fails** (returns −1) does execution continue to the fork().

Q2 (5 pt) Why does the socket communication use the port number instead of process id number to identify server or client process? 2021

ANSWER

- A **PID is assigned dynamically** by the local OS. It changes every time the server is restarted and is **unique only within one host**, so a client on another machine cannot know or rely on it.
- A **port number is fixed and well known**, tied to a **service** rather than to a particular process (e.g. HTTP = 80, SSH = 22; ports below 1024 are reserved). A client can find a service just from IP address and port. A socket is identified by **IP:port**, e.g. 161.25.19.8:1625.
- The process behind a service can change (restart, failover, a pool of worker processes or threads) and one process may offer several services. The port stays the same, and the network stack uses it to deliver packets to the right endpoint.

Q3 (8 pt) What is processor affinity? Explain why it needed to be considered during process scheduling on multiple CPU systems. 2021

ANSWER

Processor affinity: a process has an affinity for the processor it is currently running on. It is kept on (or restricted to) that same CPU rather than migrated to another one.

Why it matters on multiple-CPU systems: while a process runs, its recently used data fills that processor's **cache**. If the scheduler migrates it to another CPU, the old cache contents must be **invalidated** and the new processor's cache must be **repopulated**. That causes many cache misses and memory stalls, which is expensive. Keeping the process on the same CPU reuses the "warm" cache. On NUMA systems it also keeps the process close to its local memory.

Soft affinity: the OS tries to keep the process on the same CPU but may migrate it. **Hard affinity:** the process specifies the set of CPUs it may run on. The scheduler must balance affinity against **load balancing** (push/pull migration), which moves processes from busy to idle CPUs.

Q4 (7 pt) Explain why implementing synchronization primitives by disabling interrupts is not appropriate in a single-processor system if the synchronization primitives are to be used in user-level programs. 2021

ANSWER

Disabling interrupts works on a uniprocessor only for **trusted kernel code**. It is the wrong tool for **user-level** synchronization because:

1. **It is a privileged instruction.** Letting user programs execute it breaks dual-mode protection; the hardware traps if user mode tries.
2. **The OS loses control of the CPU.** While interrupts are off, the **timer interrupt** cannot fire, so no preemption or context switch can happen. A buggy or malicious program could disable interrupts and never re-enable them (e.g. an infinite loop in its critical section). It would then **monopolize the CPU** and hang the whole system.
3. **Other events are delayed or lost.** I/O-completion interrupts and clock ticks are postponed, so devices sit idle, the system clock drifts and response time suffers. User critical sections can be long, which makes this worse.

(It also does not scale: on a multiprocessor it only stops the local CPU.) User programs should instead use OS-provided semaphores/mutexes or atomic hardware instructions such as test_and_set or swap.

Q5 (15 pt) Discuss how the following pairs of scheduling criteria conflict in certain settings. 2021

Q5.1 CPU utilization and response time 2021

ANSWER

CPU utilization vs. response time. CPU utilization is highest when context switches are rare, as with FCFS or a long quantum, because each switch is pure overhead. Good response time for interactive users needs **frequent preemption** (a short RR quantum) so every user gets the CPU quickly. That causes many context switches, so the CPU spends more time switching and less doing useful work, which lowers utilization.

Q5.2 Average turnaround time and maximum waiting time 2021

ANSWER

Average turnaround time vs. maximum waiting time. Average turnaround is minimized by running the **shortest jobs first** (SJF/SRTF). But a long job can be postponed again and again as short jobs keep arriving, so it may **starve**. Its waiting time, the **maximum** waiting time, becomes very large (unbounded). Bounding the maximum wait (e.g. with FCFS or aging) raises the average turnaround.

Q5.3 I/O device utilization and CPU utilization 2021

ANSWER

I/O device utilization vs. CPU utilization. Keeping devices busy means running **I/O-bound** processes as soon as their I/O completes, so they can issue the next request quickly. This means frequently preempting CPU-bound processes, and the context-switch overhead lowers CPU utilization. Maximizing CPU utilization means letting CPU-bound jobs run long without switching, which leaves the I/O devices **idle** while I/O-bound jobs wait for the CPU.

Q5.4 Deadline and fairness 2021

ANSWER

Deadline vs. fairness. To meet deadlines, the scheduler must always give the CPU to the most urgent process (earliest deadline or real-time priority), regardless of how much CPU it has already had. Other processes may wait a long time or **starve**, which is unfair. A **fair** scheduler gives everyone an equal share, so urgent tasks may not get enough CPU in time and **miss their deadlines**.

Q5.5 Throughput and context switch overhead 2021

ANSWER

Throughput vs. context-switch overhead. Throughput (jobs completed per unit time) is improved by finishing short jobs quickly, which needs preemption: SJF/SRTF or a small RR quantum. Every preemption adds **context-switch overhead**, during which no useful work is done. Minimizing switches (FCFS, large quantum) traps short jobs behind long ones (**convoy effect**), so fewer jobs complete per unit time. If the quantum is too small, overhead itself eats the throughput.

6. Multilevel queue scheduling

Q6.1 (5 pt) List two possible scheduling among queues, which we discussed in class. 2021

ANSWER

Two ways to schedule **among** the queues of a multilevel queue:

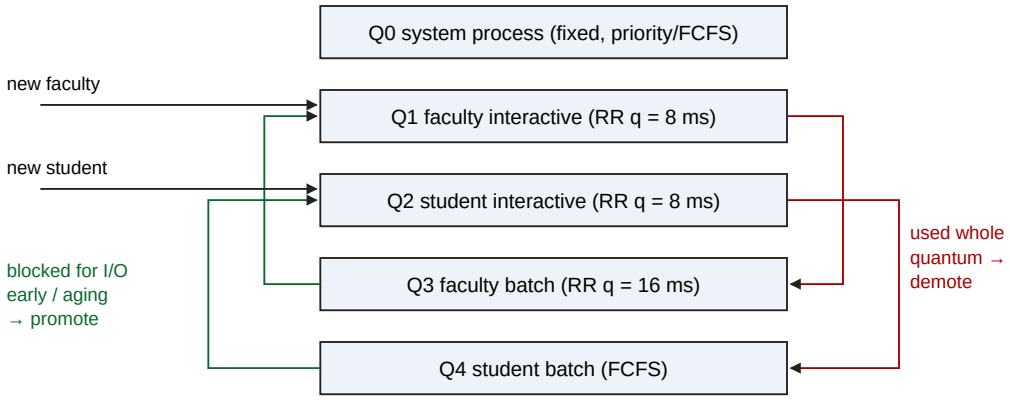
1. **Fixed-priority (preemptive) scheduling.** Serve all processes in a higher-priority queue (e.g. foreground/interactive) before any process in a lower queue (background/batch). A process arriving in a higher queue preempts a lower one. Lower queues may **starve**.
2. **Time slicing among queues.** Each queue gets a fixed share of CPU time, which it schedules among its own processes. Example: 80% to the foreground queue (RR) and 20% to the background queue (FCFS).

Q6.2 (10 pt) Modify the following multilevel queue scheduling system such that a new process will automatically enter the appropriate queue after a few runs. 2021

system process
faculty interactive process
student interactive process
faculty batch process
student batch process

ANSWER

Turn it into a **multilevel feedback queue**: a process may **move between queues** according to its observed behaviour, so it reaches the right queue automatically after a few runs.



Higher queue preempts lower (fixed priority between queues); Q0 never moves.

- A new process enters the **interactive** queue of its class (faculty → Q1, student → Q2), which uses a short quantum.

- If it **uses its whole quantum**, it is CPU-bound, so it is **demoted** to the batch queue of its class (Q1→Q3, Q2→Q4).
- If it **blocks for I/O before the quantum ends**, it is interactive, so it stays in or is **promoted** back to the interactive queue (Q3→Q1, Q4→Q2).
- **Aging** promotes processes that wait too long, preventing starvation. System processes stay fixed in Q0.

Defined by: number of queues, scheduling algorithm per queue, when to promote, when to demote, and which queue a new process enters.

7. Multilevel queue scheduling (heading as printed; the questions are about synchronization)

Q7.1 (5 pt) What is **busy-waiting** in the discussion of critical section problem? Explain how **busy-waiting** may degrade the system performance. 2021

ANSWER

Busy waiting (spinning) means a process waiting to enter its critical section stays in a loop that keeps testing a condition until it can enter, e.g. `while (test_and_set(&lock));` or `while (S <= 0);`. It never gives up the CPU while waiting.

Degraded performance: the waiting process **wastes CPU cycles** doing no useful work, and other ready processes could have used them. On a single CPU, the spinner uses its entire time slice while the lock holder cannot run to release the lock, so everyone waits longer. If the spinner has higher priority than the lock holder, the holder may never run, and the system can even **deadlock** (priority inversion). Busy waiting is acceptable only for very short waits on multiprocessors, where it avoids a context switch.

Q7.2 (10 pt) How to implement semaphore without **busy-waiting** using `block` and `wakeup`? You need to write down the definition of data structure `semaphore` and `wait` and `signal` operations. 2021

ANSWER

Each semaphore has an integer value and a **queue of waiting processes**. Instead of spinning, a process that must wait is **blocked** (moved to the waiting state and placed on the semaphore's queue). `signal` **wakes up** one waiting process (moves it to the ready queue).

```
typedef struct {
    int value;
    struct process *list;  /* queue of waiting PCBs */
} semaphore;

void wait(semaphore *S) {
    S->value--;
    if (S->value < 0) {
        add this process to S->list;
        block();               /* running -> waiting */
    }
}

void signal(semaphore *S) {
    S->value++;
    if (S->value <= 0) {
        remove a process P from S->list;
        wakeup(P);             /* waiting -> ready */
    }
}
```

If value is negative, |value| is the number of waiting processes. block() and wakeup() are system calls. wait() and signal() must still be atomic: disable interrupts inside them on a uniprocessor, or use a spinlock on a multiprocessor. That busy wait is limited to these few instructions, so it is short and rare.

Q8 (10 pt) We have discussed the correct solution to the consumer-producer problem, which does not use any synchronization mechanism, as shown below. Please explain why it can only use `BUFFER_SIZE-1` elements of buffer. 2021

```
Producer:
    while (true) {
        /* produce an item in next produced */
        while (((in + 1) % BUFFER_SIZE) == out)
            ; /* do nothing */
        buffer[in] = next_produced;
        in = (in + 1) % BUFFER_SIZE;
    }

Consumer:
    while (true) {
        while (in == out)
            ; /* do nothing */
        next_consumed = buffer[out];
        out = (out + 1) % BUFFER_SIZE;
        /* consume the item in next consumed */
    }
```

ANSWER

The solution uses only `in` (next free slot) and `out` (next full slot) to describe the buffer state, and it defines **empty** as `in == out` (consumer waits).

If the producer were allowed to fill **all** BUFFER_SIZE slots, then after the last insert `in` would wrap around and also equal `out`. **"Full" would look exactly like "empty"**, and the consumer could not tell them apart. To avoid this ambiguity, the producer treats the buffer as **full when** `(in + 1) % BUFFER_SIZE == out`, so one slot is always left unused.

Example with BUFFER_SIZE = 10: out = 0 and the producer has filled b0–b8, so in = 9. Now (9+1) % 10 = 0 = out, and the producer waits with only **9 items** stored. b9 stays empty; filling it would make in = 0 = out, which means "empty".

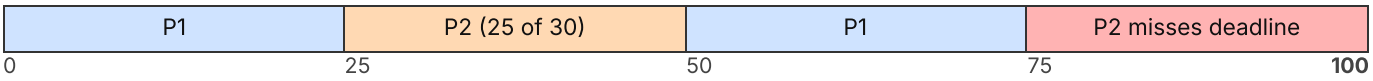
To use all N slots, add a `counter` of items (which then needs synchronization because counter++/counter-- race) or a full/empty flag.

9. Consider two processes, P_1 and P_2, where $p_1 = d_1 = 50$, $t_1 = 25$, $p_2 = d_2 = 75$, and $t_2 = 30$.

Q9.1 (10 pt) Can these two processes be scheduled using rate-monotonic scheduling (RMS)? Illustrate your answer using a Gantt chart such as the ones we did in class. 2021

ANSWER

CPU utilization U = 25/50 + 30/75 = 0.50 + 0.40 = **0.90**. The RMS guaranteed bound for N = 2 is $2(2^{1/2} - 1)$ ≈ **0.83**. Since U > 0.83, schedulability is not guaranteed, so draw the chart. RMS uses static priority by period: P1 (period 50) has **higher priority** than P2 (period 75).



- 0–25: P1 runs (deadline 50 met).
- 25–50: P2 runs, but at t = 50 P1's second period starts and, having higher priority, **preempts P2**. P2 has done only 25 of its 30 ms.
- 50–75: P1 runs (deadline 100 met).
- t = 75: P2's deadline arrives with 5 ms still left, so **P2 misses its deadline**.

No, P1 and P2 cannot be scheduled with RMS.

Q9.2 (10 pt) Illustrate the scheduling of these two processes using earliest-deadline-first (EDF) scheduling. 2021

ANSWER

EDF assigns priority dynamically: the job with the **earliest absolute deadline** runs (on a tie, the running job continues). U = 0.90 ≤ 1, so EDF can schedule them.



- t=0: P1 (deadline 50) vs P2 (deadline 75), so P1 runs 0–25. Then P2 starts at 25.
- t=50: P1's 2nd job arrives (deadline 100). P2's deadline 75 is earlier, so **P2 is not preempted** and finishes at 55 (deadline 75 met). P1 runs from 55.
- t=75: P2's 2nd job arrives (deadline 150). P1's deadline 100 is earlier, so P1 finishes at 80 (met). P2 runs 80–110.
- t=100: P1's 3rd job (deadline 150) ties with P2, so no preemption. P2 finishes at 110 (met), and P1 runs 110–135 (met). Idle 135–150.
- At t = 150 (LCM of 50 and 75) both tasks restart, so the pattern repeats.

All deadlines are met under EDF.

2022 · CS5730 Operating Systems Midterm (2022/11/22)

Essay Questions: Your answers should be as concise and clear as possible

Q1 (5 pt) Explain why implementing synchronization primitives by disabling interrupts is not appropriate in a single-processor system if the synchronization primitives are to be used in user-level programs. 2022

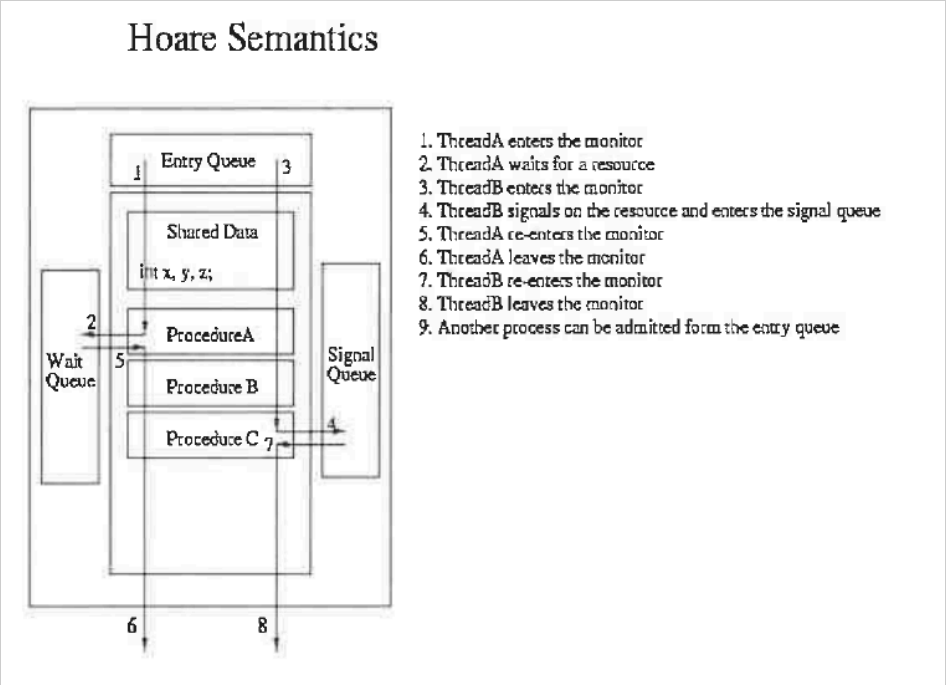
ANSWER

Disabling interrupts works on a uniprocessor only for **trusted kernel code**. It is the wrong tool for **user-level** synchronization because:

1. **It is a privileged instruction.** Letting user programs execute it breaks dual-mode protection; the hardware traps if user mode tries.
2. **The OS loses control of the CPU.** While interrupts are off, the **timer interrupt** cannot fire, so no preemption or context switch can happen. A buggy or malicious program could disable interrupts and never re-enable them (e.g. an infinite loop in its critical section). It would then **monopolize the CPU** and hang the whole system.
3. **Other events are delayed or lost.** I/O-completion interrupts and clock ticks are postponed, so devices sit idle, the system clock drifts and response time suffers. User critical sections can be long, which makes this worse.

(It also does not scale: on a multiprocessor it only stops the local CPU.) User programs should instead use OS-provided semaphores/mutexes or atomic hardware instructions such as test_and_set or swap.

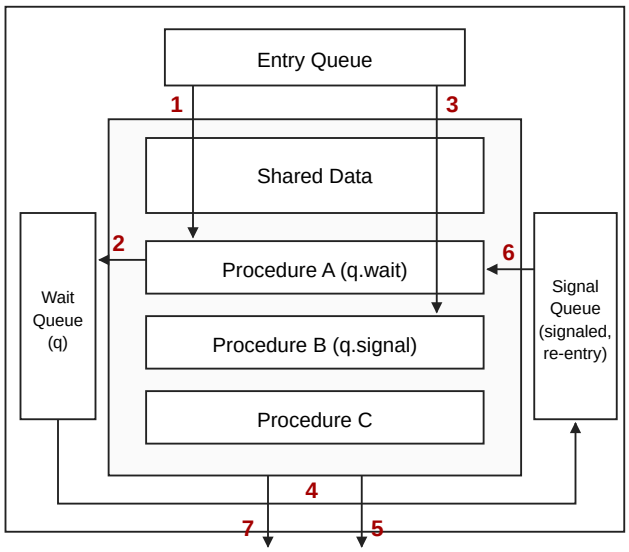
Q2 (15 pt) Note that only one process is active within the monitor. Suppose `A` associated with the condition variable `q` is resumed when `B` invokes `q.signal()`. We discussed two possibile solutions in class. One is **signal and wait**, and the flow is illustrated in the following (Hoare semantics). Please modify the Hoare semantics and depict the other solution (i.e., **signal and continue**). 2022



ANSWER

Signal-and-continue: when B executes `q.signal()`, **B keeps running** in the monitor. The waiting process A is only moved from q's wait queue to a ready/re-entry queue (the "signal queue" now holds the *signaled*

process). A resumes after B leaves the monitor or waits.



1. Thread A enters the monitor
2. A waits on q → q's wait queue (monitor released)
3. Thread B enters the monitor
4. B signals q: A is moved from wait queue → signal queue; **B CONTINUES running**
5. B leaves the monitor
6. A re-enters the monitor (signal queue has priority over entry queue)
7. A leaves the monitor
8. Another process can be admitted from the entry queue

Difference from Hoare: signal queue now holds the SIGNALED process, not the signaler; 8 steps instead of 9.

Compared with Hoare: in Hoare's version A runs immediately and the signaler B waits in the signal queue. In signal-and-continue, A may find the condition changed by the time it runs, so waiters should re-check it with `while (!cond) q.wait();`. There are fewer context switches, and this is what Java and Pthreads use.

Q3 (20 pt) In class, we have discussed implementing monitor with **signal and wait** via semaphores as follows. 2022

```
Variables
    semaphore mutex;    // (initially = 1)
    semaphore next;     // (initially = 0)
    int next-count = 0;

Each procedure F will be replaced by
    wait(mutex);
        ...
        // body of F;
        ...
    if (next_count > 0)
        signal(next)
    else
        signal(mutex);

For each condition variable x,
    semaphore x_sem;    // (initially = 0)
    int x-count = 0;

The operation x.wait can be implemented as:
    x-count++;
    if (next_count > 0)
        signal(next);
    else
        signal(mutex);
    wait(x_sem);
    x-count--;

The operation x.signal can be implemented as:
    if (x-count > 0) {
        next_count++;
        signal(x_sem);
        wait(next);
        next_count--;
    }
```

Please modify the above code to implement **signal and continue**.

ANSWER

In signal-and-wait the **signaler** blocks on `next`. For **signal-and-continue**, the signaler must **not** block. Instead, the **signaled** process waits on `next` to re-enter after the signaler leaves. Changed lines are marked NEW:

```
Variables
    semaphore mutex;      // (initially = 1)
    semaphore next;       // (initially = 0)
                          // = signaled procs
                          //   waiting to re-enter
    int next_count = 0;

Each procedure F (unchanged)
    wait(mutex);
        ...
        // body of F;
        ...
    if (next_count > 0)
        signal(next);     // signaled proc first
    else
        signal(mutex);
```

```
For each condition variable x
    semaphore x_sem;      // (initially = 0)
    int x_count = 0;

x.wait
    x_count++;
    if (next_count > 0)
        signal(next);
    else
        signal(mutex);
    wait(x_sem);          // sleep until signaled
    wait(next);           // NEW: wait until the
                          //   signaler leaves/waits
    next_count--;         // NEW: back in monitor
                          // (x_count-- moved to
                          //   x.signal)

x.signal                  // signal-and-continue
    if (x_count > 0) {
        x_count--;        // NEW: done by signaler
        next_count++;
        signal(x_sem);    // wake one waiter
        // REMOVED: wait(next); next_count--;
    }                     // signaler continues
```

Why: the signaler no longer suspends itself. The woken process must re-acquire the monitor, and it gets it through `next`, ahead of the entry queue, when the signaler exits or waits. x_count is decremented by the signaler, so two consecutive signals cannot wake the same single waiter twice. Callers should use `while (!B) x.wait();`.

Q4 (10 pt) Suppose a architecture use 48-bit addressing with 4K paging on 64-processors. How much memory space a process may have? How many entries a page table may have? How should a page table be stored in memory? 2022

ANSWER

Memory space per process: 48-bit addresses give 2^{48} bytes = **256 TB** of logical address space.

Page-table entries: 4 KB = 2^{12} gives a 12-bit offset, so the page number is 48 − 12 = 36 bits. That makes **2^{36} ≈ 68.7 billion (64 G) entries**. On a 64-bit processor each entry is 8 bytes, so a single flat table would need $2^{36} \times 8 = 2^{39}$ B = **512 GB per process**.

How to store it: a flat table is far too large to keep, let alone contiguously. **Page the page table** with a **multi-level (hierarchical) page table**. x86-64 uses **4 levels**:

level 4: 9 bits	level 3: 9 bits	level 2: 9 bits	level 1: 9 bits	offset: 12 bits

Each table at each level has 2^9 = 512 entries × 8 B = 4 KB, exactly one page, so tables are paged themselves. **Only the tables for parts of the address space actually in use are allocated**, which keeps the memory cost small. Alternatives: a **hashed page table** (good for sparse 64-bit spaces) or an **inverted page table** (one entry per physical frame). A **TLB** hides the extra memory accesses.

5. Multilevel queue scheduling

Q5.1 (5 pt) List two possible scheduling among queues, which we discussed in class. 2022

ANSWER

Two ways to schedule **among** the queues of a multilevel queue:

1. **Fixed-priority (preemptive) scheduling.** Serve all processes in a higher-priority queue (e.g. foreground/interactive) before any process in a lower queue (background/batch). A process arriving in a

higher queue preempts a lower one. Lower queues may **starve**.

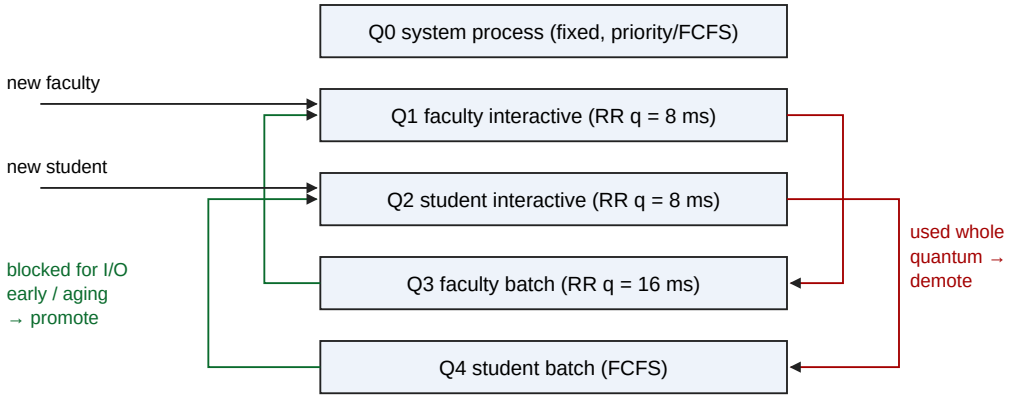
2. **Time slicing among queues.** Each queue gets a fixed share of CPU time, which it schedules among its own processes. Example: 80% to the foreground queue (RR) and 20% to the background queue (FCFS).

Q5.2 (5 pt) Modify the following multilevel queue scheduling system such that a new process will automatically enter the appropriate queue after a few runs. 2022

system process
faculty interactive process
student interactive process
faculty batch process
student batch process

ANSWER

Turn it into a **multilevel feedback queue**: a process may **move between queues** according to its observed behaviour, so it reaches the right queue automatically after a few runs.



Higher queue preempts lower (fixed priority between queues); Q0 never moves.

- A new process enters the **interactive** queue of its class (faculty → Q1, student → Q2), which uses a short quantum.
- If it **uses its whole quantum**, it is CPU-bound, so it is **demoted** to the batch queue of its class (Q1→Q3, Q2→Q4).
- If it **blocks for I/O before the quantum ends**, it is interactive, so it stays in or is **promoted** back to the interactive queue (Q3→Q1, Q4→Q2).
- **Aging** promotes processes that wait too long, preventing starvation. System processes stay fixed in Q0.

Defined by: number of queues, scheduling algorithm per queue, when to promote, when to demote, and which queue a new process enters.

6. Multilevel queue scheduling (heading as printed)

Q6.1 (5 pt) What is **busy-waiting** in the discussion of critical section problem? Explain how **busy-waiting** may degrade the system performance. 2022

ANSWER

Busy waiting (spinning) means a process waiting to enter its critical section stays in a loop that keeps testing a condition until it can enter, e.g. `while (test_and_set(&lock));` or `while (S <= 0);`. It never gives up the CPU while waiting.

Degraded performance: the waiting process **wastes CPU cycles** doing no useful work, and other ready processes could have used them. On a single CPU, the spinner uses its entire time slice while the lock holder cannot run to release the lock, so everyone waits longer. If the spinner has higher priority than the lock holder, the holder may never run, and the system can even **deadlock** (priority inversion). Busy waiting is acceptable only for very short waits on multiprocessors, where it avoids a context switch.

Q6.2 (5 pt) How to implement semaphore without **busy-waiting** using `block` and `wakeup`? You need to write down the definition of data structure `semaphore` and `wait` and `signal` operations. 2022

ANSWER

Each semaphore has an integer value and a **queue of waiting processes**. Instead of spinning, a process that must wait is **blocked** (moved to the waiting state and placed on the semaphore's queue). `signal` **wakes up** one waiting process (moves it to the ready queue).

```
typedef struct {
    int value;
    struct process *list;  /* queue of waiting PCBs */
} semaphore;

void wait(semaphore *S) {
    S->value--;
    if (S->value < 0) {
        add this process to S->list;
        block();               /* running -> waiting */
    }
}

void signal(semaphore *S) {
    S->value++;
    if (S->value <= 0) {
        remove a process P from S->list;
        wakeup(P);             /* waiting -> ready */
    }
}
```

If value is negative, |value| is the number of waiting processes. block() and wakeup() are system calls. wait() and signal() must still be atomic: disable interrupts inside them on a uniprocessor, or use a spinlock on a multiprocessor. That busy wait is limited to these few instructions, so it is short and rare.

Q7 (10 pt) We have discussed the correct solution to the consumer-producer problem, which does not use any synchronization mechanism, as shown below. Please explain why it can only use `BUFFER_SIZE-1` elements of buffer. 2022

```
Producer:
    while (true) {
        /* produce an item in next produced */
        while (((in + 1) % BUFFER_SIZE) == out)
            ; /* do nothing */
        buffer[in] = next_produced;
        in = (in + 1) % BUFFER_SIZE;
    }

Consumer:
    while (true) {
        while (in == out)
            ; /* do nothing */
        next_consumed = buffer[out];
        out = (out + 1) % BUFFER_SIZE;
        /* consume the item in next consumed */
    }
```

ANSWER

The solution uses only `in` (next free slot) and `out` (next full slot) to describe the buffer state, and it defines **empty** as `in == out` (consumer waits).

If the producer were allowed to fill **all** BUFFER_SIZE slots, then after the last insert `in` would wrap around and also equal `out`. **"Full" would look exactly like "empty"**, and the consumer could not tell them apart. To avoid this ambiguity, the producer treats the buffer as **full when** `(in + 1) % BUFFER_SIZE == out`, so one slot is always left unused.

Example with BUFFER_SIZE = 10: out = 0 and the producer has filled b0–b8, so in = 9. Now (9+1) % 10 = 0 = out, and the producer waits with only **9 items** stored. b9 stays empty; filling it would make in = 0 = out, which means "empty".

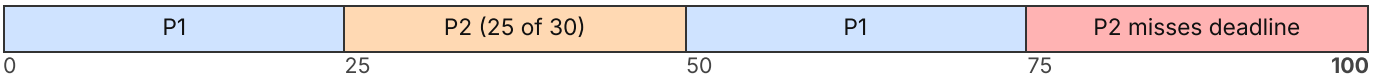
To use all N slots, add a `counter` of items (which then needs synchronization because counter++/counter-- race) or a full/empty flag.

8. Consider two processes, P_1 and P_2, where $p_1 = d_1 = 50$, $t_1 = 25$, $p_2 = d_2 = 75$, and $t_2 = 30$.

Q8.1 (10 pt) Can these two processes be scheduled using rate-monotonic scheduling (RMS)? Illustrate your answer using a Gantt chart such as the ones we did in class. 2022

ANSWER

CPU utilization U = 25/50 + 30/75 = 0.50 + 0.40 = **0.90**. The RMS guaranteed bound for N = 2 is $2(2^{1/2} - 1)$ ≈ **0.83**. Since U > 0.83, schedulability is not guaranteed, so draw the chart. RMS uses static priority by period: P1 (period 50) has **higher priority** than P2 (period 75).



- 0–25: P1 runs (deadline 50 met).
- 25–50: P2 runs, but at t = 50 P1's second period starts and, having higher priority, **preempts P2**. P2 has done only 25 of its 30 ms.
- 50–75: P1 runs (deadline 100 met).
- t = 75: P2's deadline arrives with 5 ms still left, so **P2 misses its deadline**.

No, P1 and P2 cannot be scheduled with RMS.

Q8.2 (10 pt) Illustrate the scheduling of these two processes using earliest-deadline-first (EDF) scheduling. 2022

ANSWER

EDF assigns priority dynamically: the job with the **earliest absolute deadline** runs (on a tie, the running job continues). U = 0.90 ≤ 1, so EDF can schedule them.



- t=0: P1 (deadline 50) vs P2 (deadline 75), so P1 runs 0–25. Then P2 starts at 25.
- t=50: P1's 2nd job arrives (deadline 100). P2's deadline 75 is earlier, so **P2 is not preempted** and finishes at 55 (deadline 75 met). P1 runs from 55.
- t=75: P2's 2nd job arrives (deadline 150). P1's deadline 100 is earlier, so P1 finishes at 80 (met). P2 runs 80–110.
- t=100: P1's 3rd job (deadline 150) ties with P2, so no preemption. P2 finishes at 110 (met), and P1 runs 110–135 (met). Idle 135–150.
- At t = 150 (LCM of 50 and 75) both tasks restart, so the pattern repeats.

All deadlines are met under EDF.